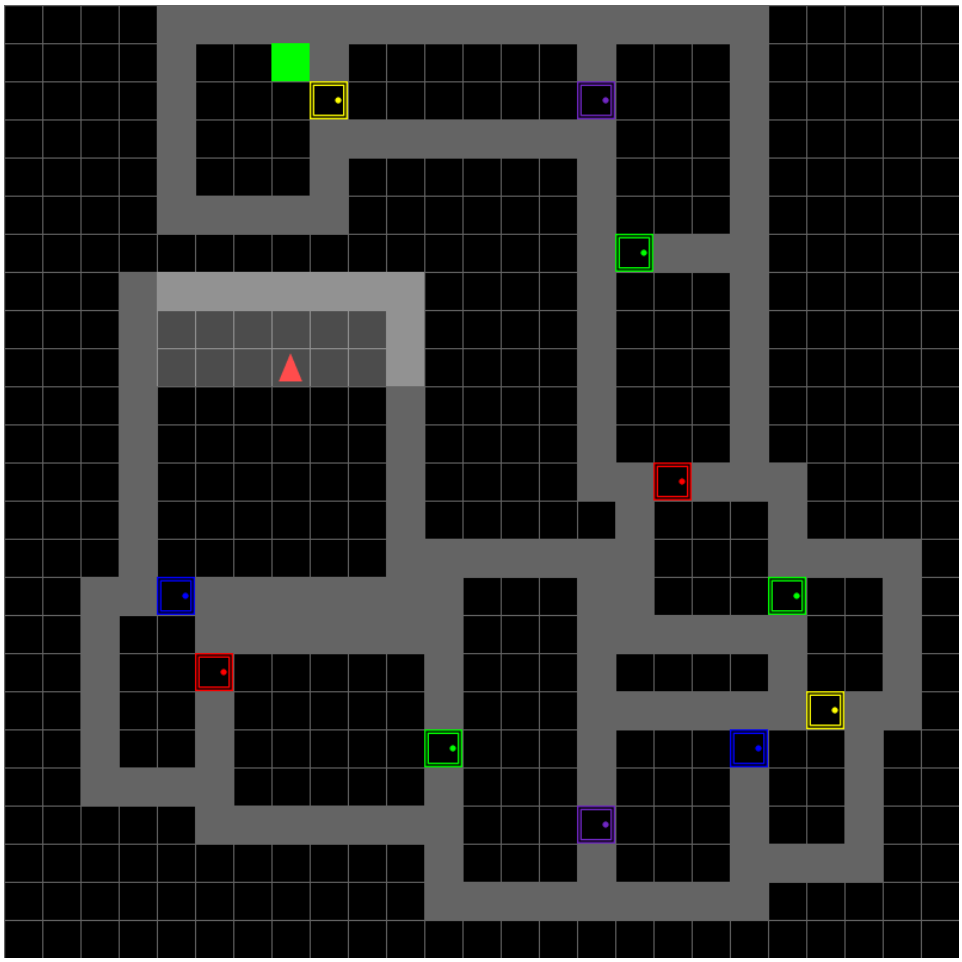


By Adi Zur & Efi Pecani

Introduction

In this project, our goal was to develop deep reinforcement learning agents capable of solving the MultiRoom environment from the MiniGrid library.
The MultiRoom environment presents a unique challenge due to its partial observability and multi-room navigation requirements, which demand effective perception and decision-making abilities. We implemented and compared two state-of-the-art approaches: Dueling Double DQN (D3QN) and Proximal Policy Optimization (PPO), analyzing their performance across environments of increasing complexity.

1. Algorithm Selection and Explanation

We implemented two different algorithms to solve the MultiRoom environment, allowing us to compare their effectiveness and characteristics.

1.1 Dueling Double DQN (D3QN)

D3QN is an off-policy value-based algorithm that enhances the original DQN architecture with two key improvements:

1. **Double Q-learning**: Uses two networks (online and target) to reduce overestimation bias. The online network selects actions while the target network evaluates them.
2. **Dueling Architecture**: Separates state value estimation (V(s)) from advantage estimation (A(s,a)), enabling better evaluation of states where action choice is less critical.

This algorithm is particularly well-suited for the MultiRoom task because:

- It efficiently handles large state spaces by focusing learning on states where actions matter.
- The dueling architecture provides better action differentiation, which is crucial for navigation tasks.
- It offers stable value estimation across the complex multi-room layout.

1.2 Proximal Policy Optimization (PPO)

PPO is an on-policy policy gradient method that introduces a clipped objective

$$L^{CLIP}(\theta) = \hat{E}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1$$

function to restrict policy update sizes:

Where:

- $r_t(\theta)$ is the probability ratio between new and old policies
- $\hat{A}_t$ is the estimated advantage
- ϵ is a hyperparameter controlling the clipping boundary (typically 0.1-0.2)

PPO is advantageous for this environment because:

- It offers excellent sample efficiency by making multiple optimization passes over trajectory data
- Its clipping mechanism prevents destructively large policy updates
- It naturally balances exploration and exploitation through entropy regularization
- It enables effective curriculum learning across environments of increasing complexity

1.3 Neural Network Architectures

D3QN Architecture: Conv2D(input(1,14,14) → 32 filters, 3×3 kernel) → ReLU

Conv2D(32 →64 filters, 3×3 kernel) → ReLU

Conv2D(64 → 128 filters, 3×3 kernel) → ReLU

Flatten

[Split into Value and Advantage Streams]

Value: FC(8192 → 256) → ReLU → FC(256 → 1)

Advantage: FC(8192 → 256) → ReLU → FC(256 → n_actions)

Q = Value + (Advantage - mean(Advantage))

PPO Architecture: Conv2D(input(3,56,56) → 32 filters, 3×3 kernel) → ReLU

Conv2D(32 → 64 filters, 3×3 kernel) → ReLU

Conv2D(64 → 64 filters, 3×3 kernel) → ReLU

Flatten

FC(flattened → 256) → ReLU

FC(256 → 128) → ReLU

[Split into Actor and Critic heads]

Actor: FC(128 → n_actions)

Critic: FC(128 → 1)

The D3QN architecture works with smaller (14×14) preprocessed inputs, while the PPO architecture processes larger (56×56) images. Both use convolutional layers to extract spatial features crucial for navigation but differ in their output mechanisms according to their respective algorithms.

1.4 Key Hyperparameters

Algorithm	Parameter
D3QN	Learning Rate
	Gamma (discount)
	Epsilon Decay
	Buffer Size
	Target Update
PPO	Learning Rate
	Gamma (discount)
	Clip Ratio ( )
	Entropy Coefficient
	Update Epochs

(Fig. 1: Epsilon decay rate)

2. Preprocessing and Exploration-Exploitation

2.1 Preprocessing

D3QN Preprocessing:

1. **Green Channel Extraction**: Since the target is green, we extracted only the green channel from RGB inputs

2. **Downsampling**: Resized images to 14×14 pixels (from 56×56) for faster computation
3. **Normalization**: Rescaled pixel values to [0,1] range for numerical stability

PPO Preprocessing:

1. **Dimension Reordering**: Converted images from HWC (Height, Width, Channel) to CHW format for PyTorch
2. **Normalization**: Rescaled pixel values to [0,1] range
3. **No Downsampling**: Retained the original 56×56 resolution to preserve spatial information

2.2 Exploration-Exploitation Management

D3QN Approach:

- **Epsilon-Greedy Strategy**: Started with high exploration (=1.0).
- **Decay Schedule**: Used a slow decay rate (0.99995) to maintain exploration
- **Action Penalties**: Discouraged repeated actions to prevent getting stuck

PPO Approach:

- **Entropy Regularization**: Added entropy bonus to the loss function, increasing for larger environments:
  - Small environment: 0.05 entropy coefficient
  - Medium environment: 0.1 entropy coefficient
  - Large environment: 0.15 entropy coefficient
- **Curriculum Learning**: Started with simpler environments, transferring knowledge to more complex ones
- **Learning Rate Annealing**: This is progressively decreased through curriculum stages

2.3 Reward Shaping

Both algorithms benefited significantly from custom reward shaping to accelerate learning:

D3QN Reward Structure:

- Exploration reward: +0.05
- Moving forward after opening door: +0.2
- Opening new door: +0.5
- Reaching target: +1 (small) → +2 (medium) → +3 (large)
- Penalty for unnecessary door opening: -0.3
- Penalty for repeated toggling: -0.2
- Penalty for action repetition: -0.1

PPO Reward Structure:

- Step penalty: -0.001 (encouraging efficiency)
- Door bonus: +5.0 (only for new rooms)
- Goal bonus: +15.0 (increased for larger environments)

The PPO implementation included a smart reward wrapper that tracked visited rooms to prevent "door cycling" behavior, where the agent would repeatedly cross the same door to farm rewards.

3. Performance and Analysis

3.1 Training Progression

D3QN Results:

- **Small Environment (N2-S4)**: 100% success rate after 60,000 episodes, avg. length 7.3 steps
- **Medium Environment (N4-S5)**: 100% success rate after 80,000 episodes, avg. length 33.8 steps
- **Large Environment (N6-v0)**: 90% success rate after 90,000 episodes, avg. length 59.5 steps

(See Fig 2, 3, 4)

PPO Results:

- **Small Environment (N2-S4)**: 98% success rate after ~6,700 episodes, avg. length 14.5 steps

- **Medium Environment (N4-S5)**: ~95% success rate after ~5,000 episodes,
  avg. length ~45 steps
- **Large Environment (N6-v0)**: 99% success rate after ~5,500 episodes, avg. length 94.7 steps

(See Fig. 5,6,7)

3.2 Comparative Analysis

Sample Efficiency:

- PPO required significantly fewer episodes to reach high performance (6,700 vs 60,000 for small environment)
- D3QN showed more stable but slower learning curves

Final Performance:

- Both algorithms achieved excellent final performance (90-100% success)
- D3QN found more efficient paths in smaller environments (7.3 vs 14.5 steps)
- PPO performed better in the large environment (99% vs 90% success)

Training Stability:

- D3QN required careful tuning of epsilon decay and reward shaping
- PPO exhibited more consistent improvement through entropy-based exploration
- Both algorithms benefited from curriculum learning approaches

Action Distribution:

- D3QN initially showed a more balanced
  action selection between movement and turning.
- PPO developed a preference for forward movement with strategic turns and toggles.
- Both algorithms learned to use the toggle action effectively for doors.

(Fig. 8: Action distribution comparison)

3.3 Key Insights and Challenges

Common Challenges:

1. **Partial Observability**: Both algorithms needed to develop an implicit memory of the room layout

2. **Reward Sparsity**: Original environment rewards were insufficient, requiring custom shaping
3. **Door Cycling**: Agents initially learned to exploit door rewards rather than seek the goal
   Therefore got "stuck" for a while in on the verge of the entry to a new room.
   Rewards solved the problem by giving the agents a stimulus to explore new rooms.

Algorithm-Specific Insights:

- **D3QN**:
  - Smaller architecture with preprocessed inputs was computationally efficient
  - Learning was more sensitive to hyperparameter selection
  - Required significantly more training episodes
- **PPO**:
  - The curriculum learning approach transferred knowledge very effectively.
  - Entropy-based exploration adapted well to increasing environment complexity
  - Multiple optimization passes per rollout improved sample efficiency.

4. Conclusion and Discussion

We have managed in our project to implement successfully and compare two powerful deep reinforcement learning approaches for solving the MultiRoom navigation task.
Both D3QN and PPO achieved good performance (More than 90%), with some notable differences:

D3QN Strengths:

- More efficient paths in smaller environments.
- Simpler preprocessing with downsampled inputs.
- Explicit separation of state value and action advantages.

PPO Strengths:

- Superior sample efficiency (fewer training episodes significantly).
- Better performance in the largest environment.
- More effective knowledge transfer between environments.

The most critical factors for success in both approaches were:

1. **Effective Reward Shaping**: Custom rewards that encouraged exploration and goal-seeking behavior
2. **Curriculum Learning**: Progressive training from simple to complex environments
3. **Balanced Exploration**: Maintaining appropriate exploration throughout training
4. **Architecture Design**: CNN effectively processes visual observations.

Future work could explore combining the strengths of both approaches, perhaps through:

- Incorporating aspects of the dueling architecture into PPO.
- Applying PPO's efficient optimization to DQN's value estimation.
- Exploring recurrent architectures for better handling of partial observability.
- Investigating multi-task learning across different and similar MiniGrid environments.

Eventually, our long and thorough experimantation with various methods and logics, while some were rule-based and others computational approaches, got us to achieve in both RL algorithms pretty good results in solving complex navigation tasks, with partial observability in randomly created labrinths.

References & Bibliography

1. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.
2. Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., & Freitas, N. (2016).
   Dueling network architectures for deep reinforcement learning. International conference on machine learning, 1995-2003.
3. Van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double q-learning. AAAI conference on artificial intelligence, 30(1).
4. Chevalier-Boisvert, M., Willems, L., & Pal, S. (2018). Minimalistic gridworld environment for OpenAI Gym. GitHub repository, https://github.com/maximecb/gym-minigrid.

5. Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009). Curriculum learning. In Proceedings of the 26th Annual International Conference on Machine Learning (pp. 41-48).
6. RL Starter Files - https://github.com/lcswillems/rl-starter-files (Inspiration only).
7. Yi-Pei Chan Deep Reinforcement Learning for MiniGrid https://yipeichan.github.io/AdvML_deepRL.pdf (Inspiration only)

Appendix

Google Drive & Colab Links

- Research & Experimentation:

  RL_Final_Project_MiniGrid_PPO_Agent_307765230_ 307915074.ipynb

  RL_Final_Project_MiniGrid_D3QN_307765230_307915074.ipynb

- Evaluation:

  EVALUATION_NOTEBOOK_PPO_307765230_ 307915074.ipynb

  RL_Final_Project_MiniGrid_D3QN_307765230_307915074_Evaluation_Notebook.ipynb

  Video Links:

  RL-Final-Project-MiniGrid

Figures Graphics and Metrics

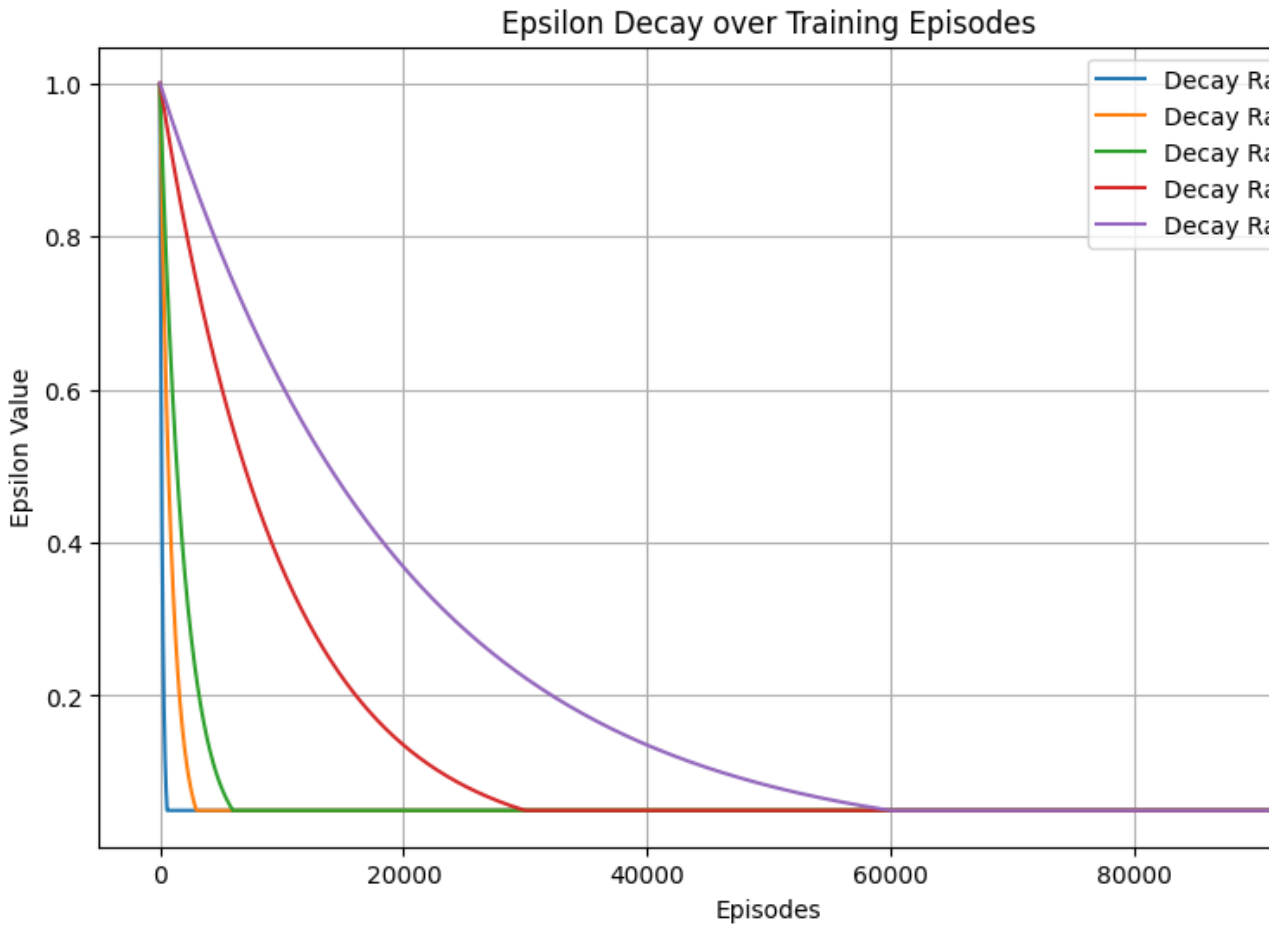


Figure 1: Epsilon decay rated over training episodes - D3QN Agent

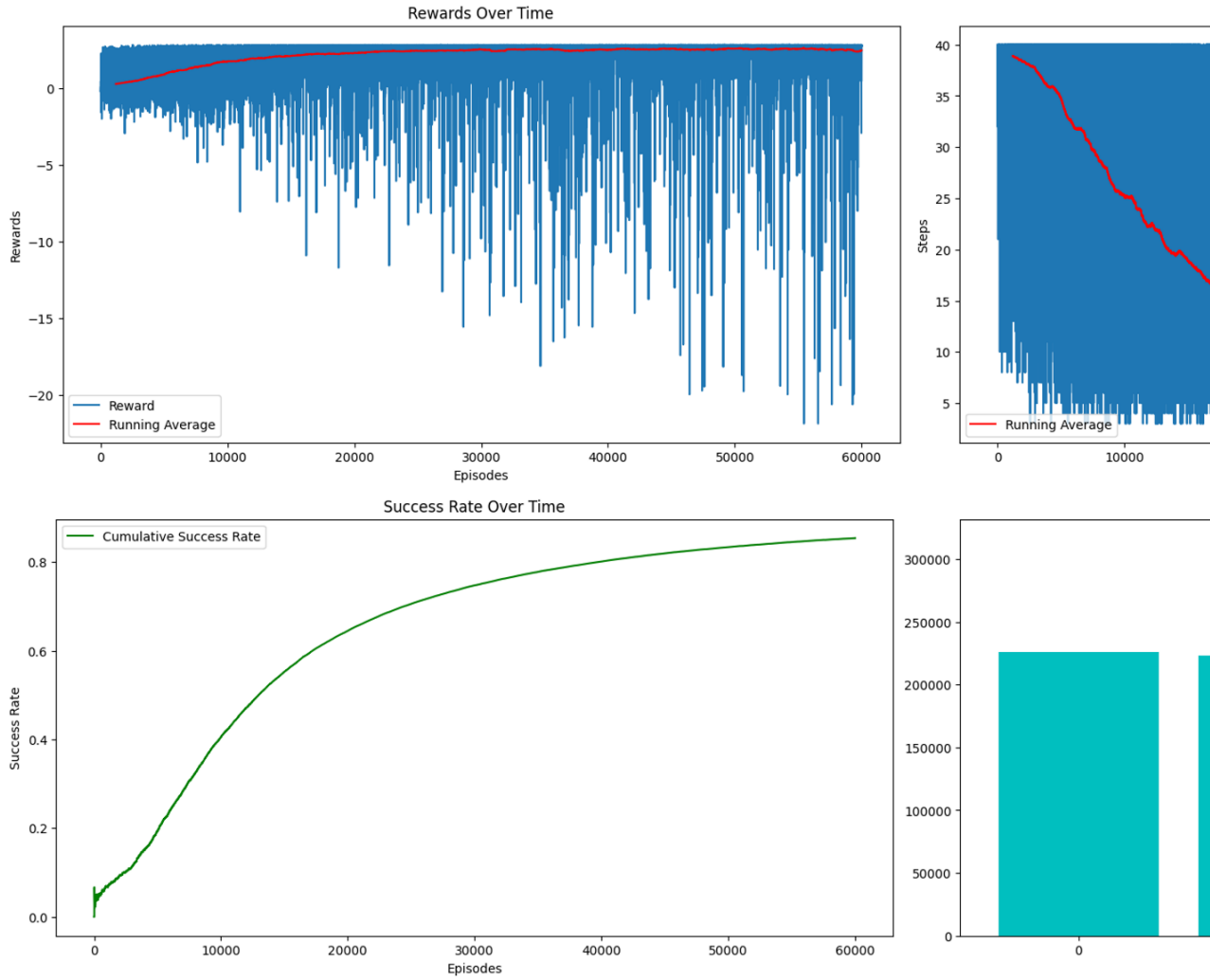


Figure 2 : Training Analysis – Small Environment - D3QN Agent

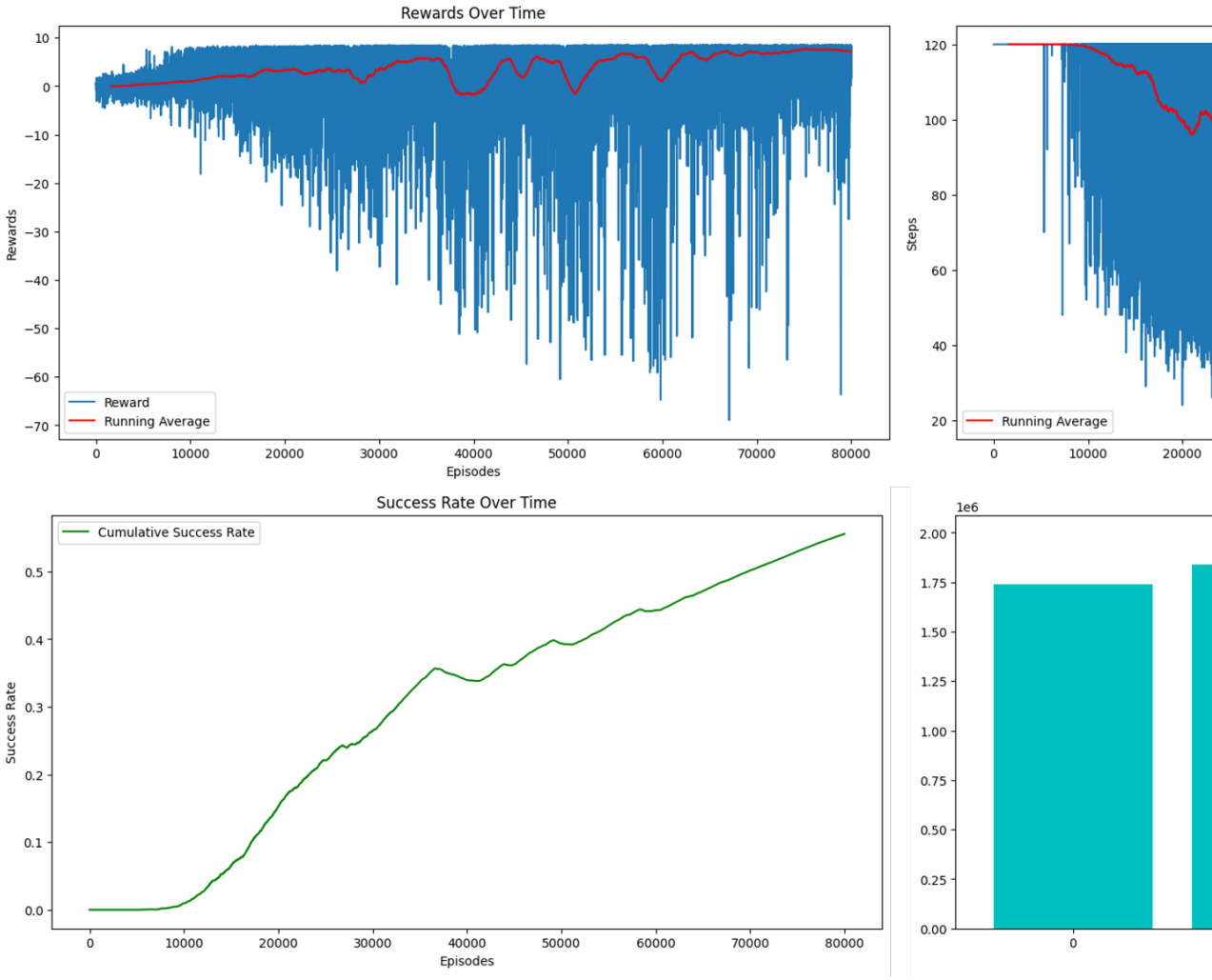


Figure 3 : Training Analysis – Medium Environment - D3QN Agent

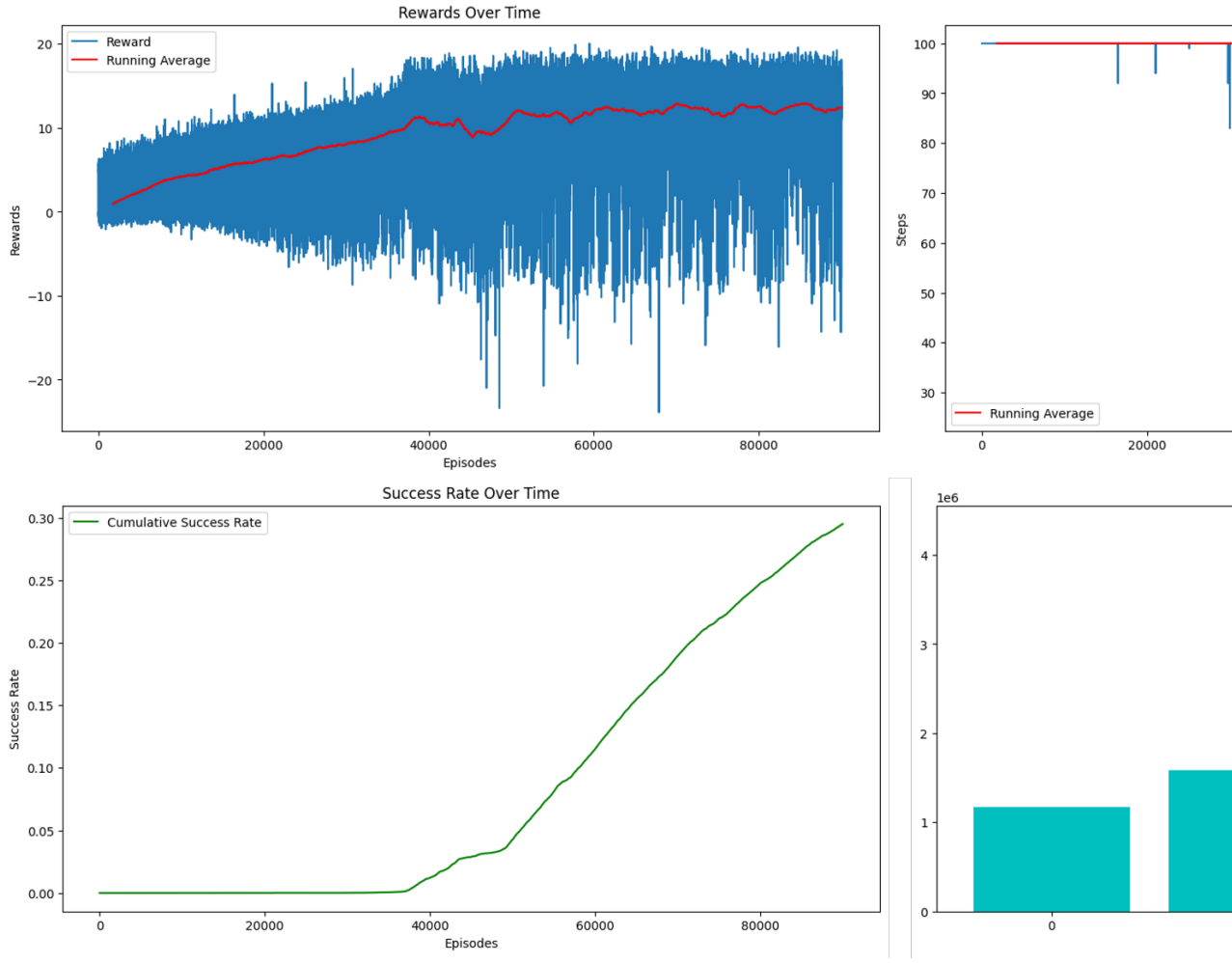


Figure 4 : Training Analysis – Large Environment - D3QN Agent

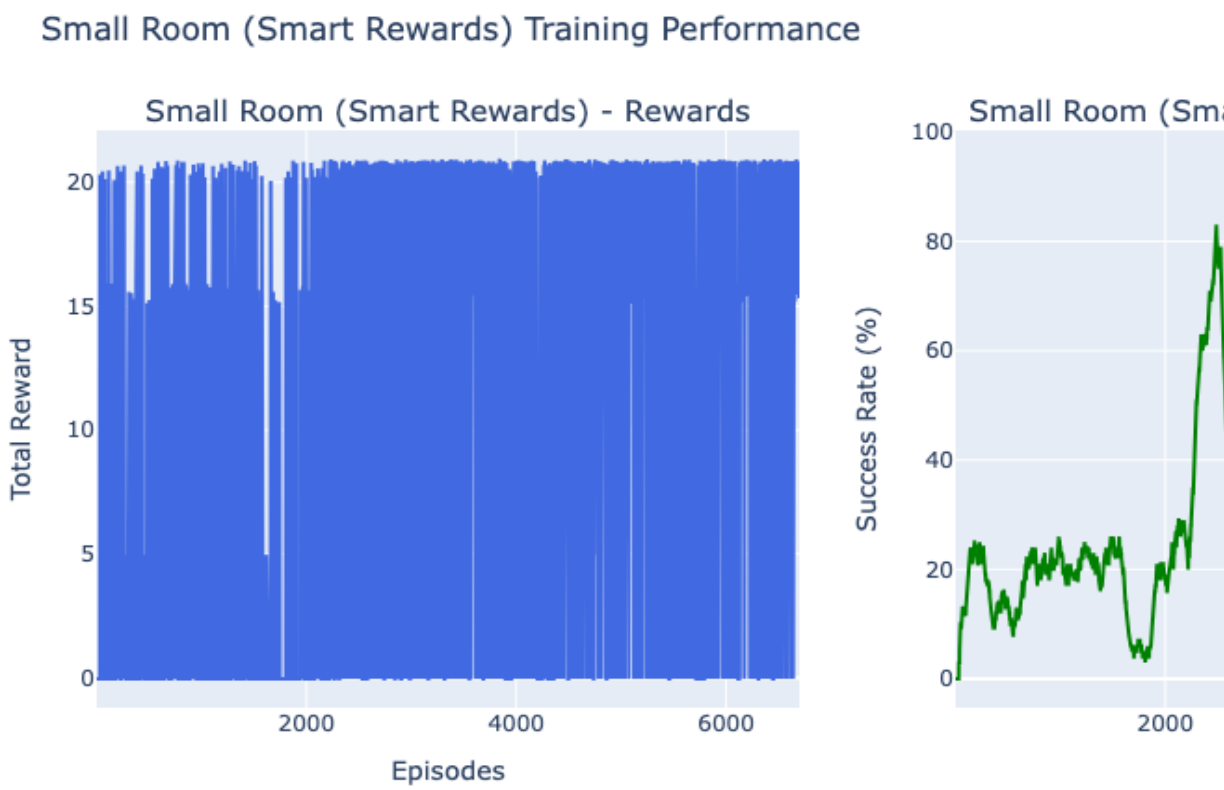


Figure 5 : Training Analysis – Small Environment - PPO Agent

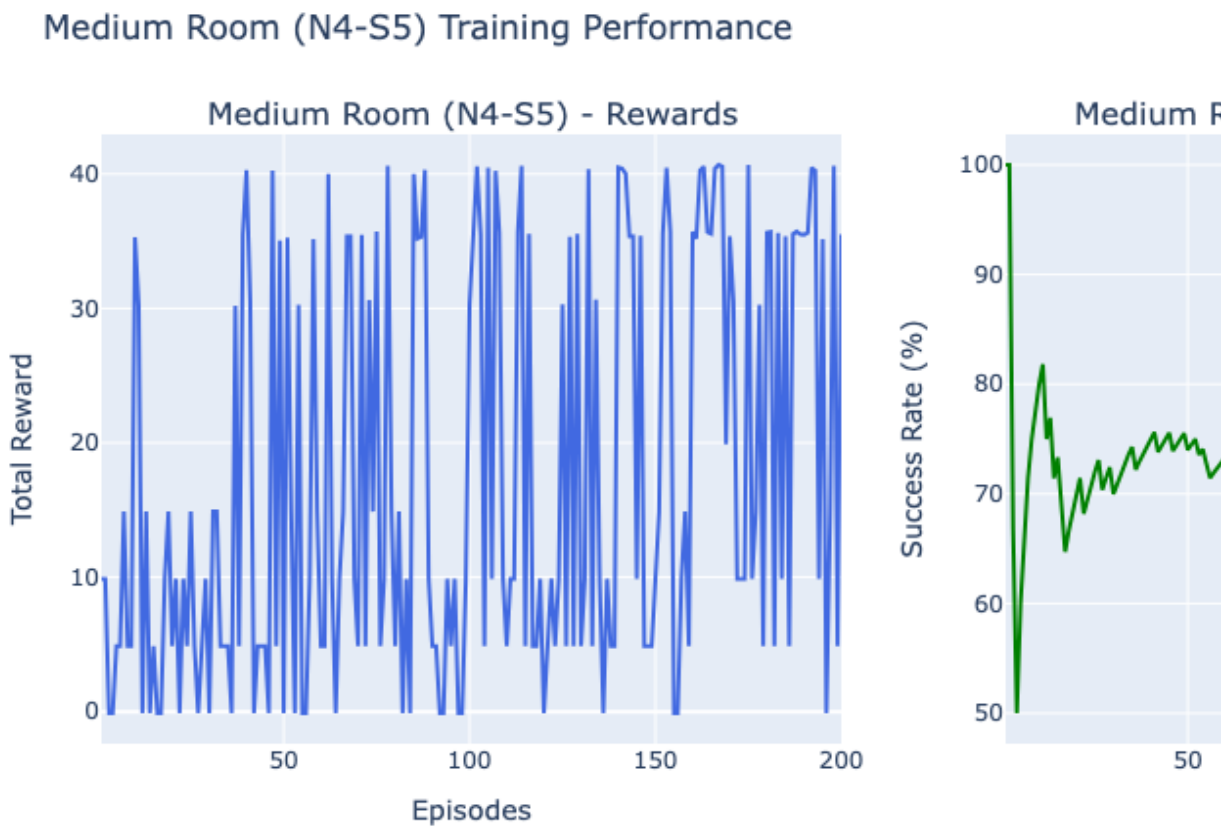


Figure 6 : Training Analysis – Medium Environment - PPO Agent

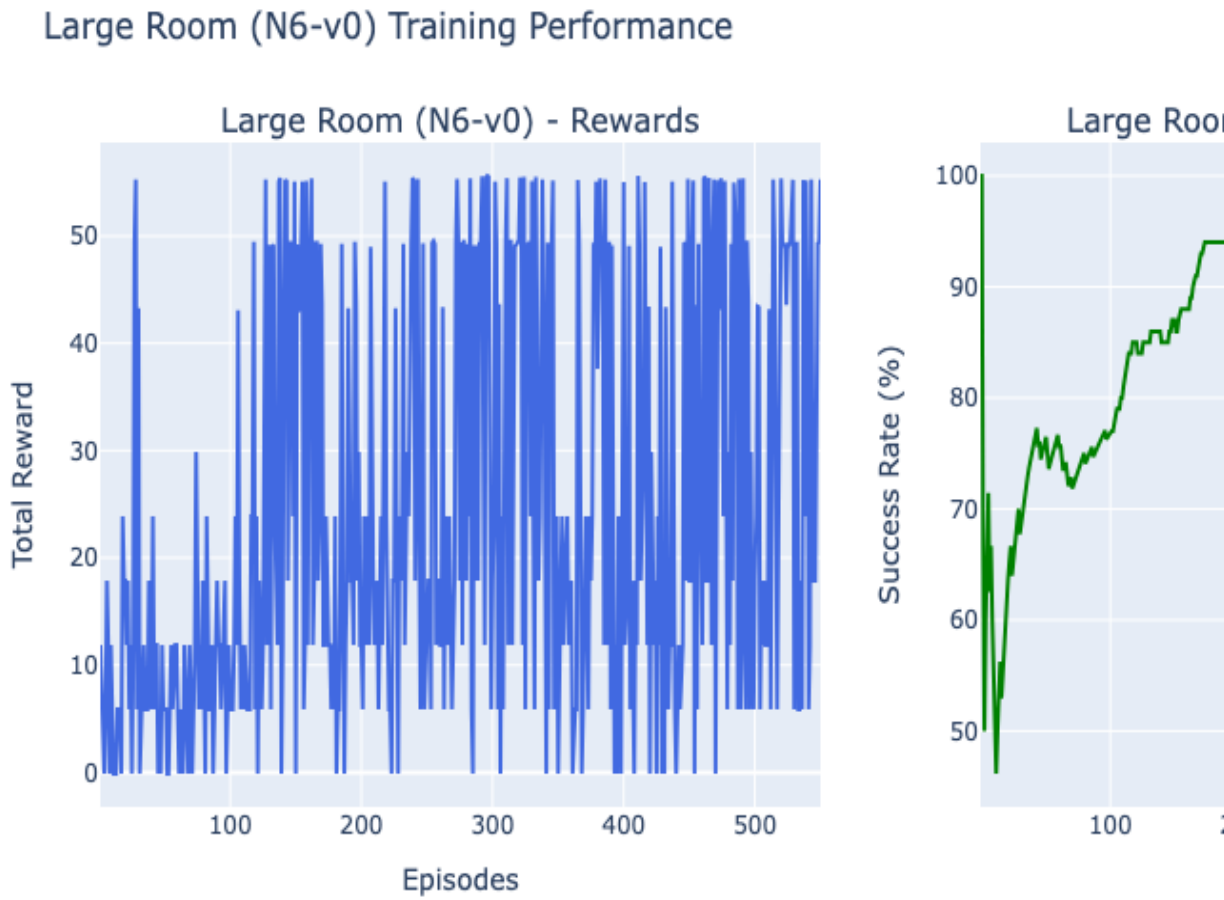


Figure 7: Training Analysis – Large Environment - PPO Agent

Fig. 8: Action Distribution Comparison - PPO Agent

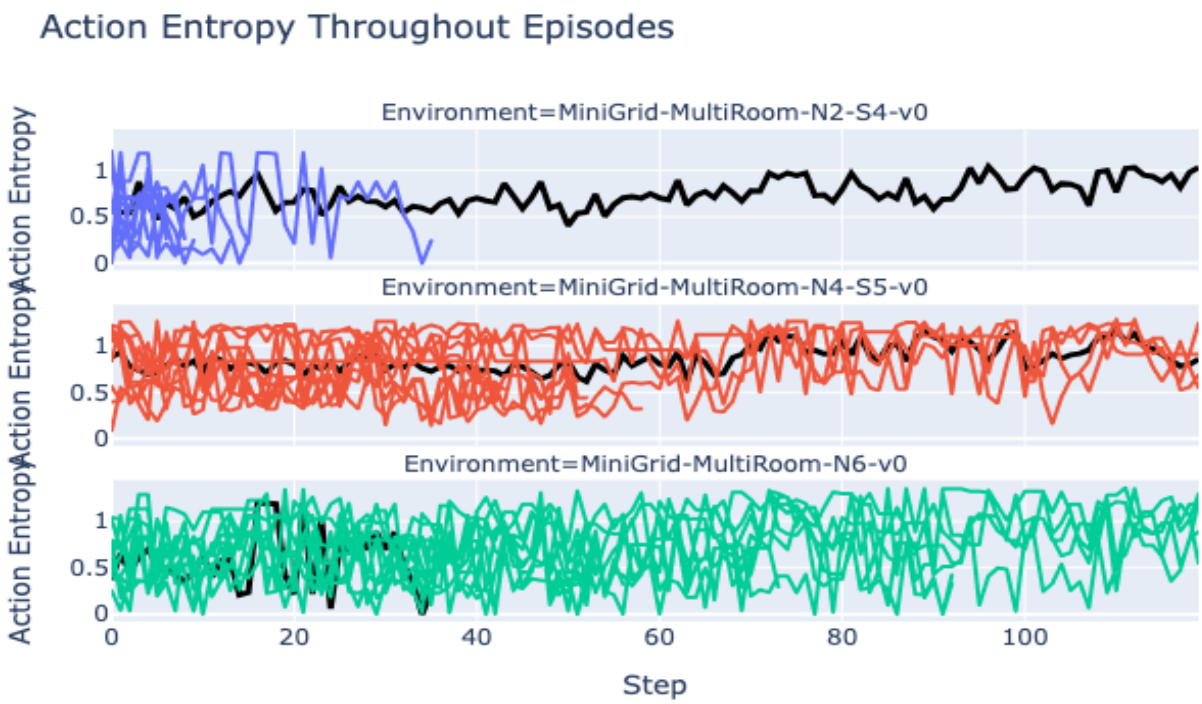
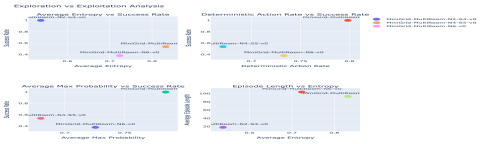
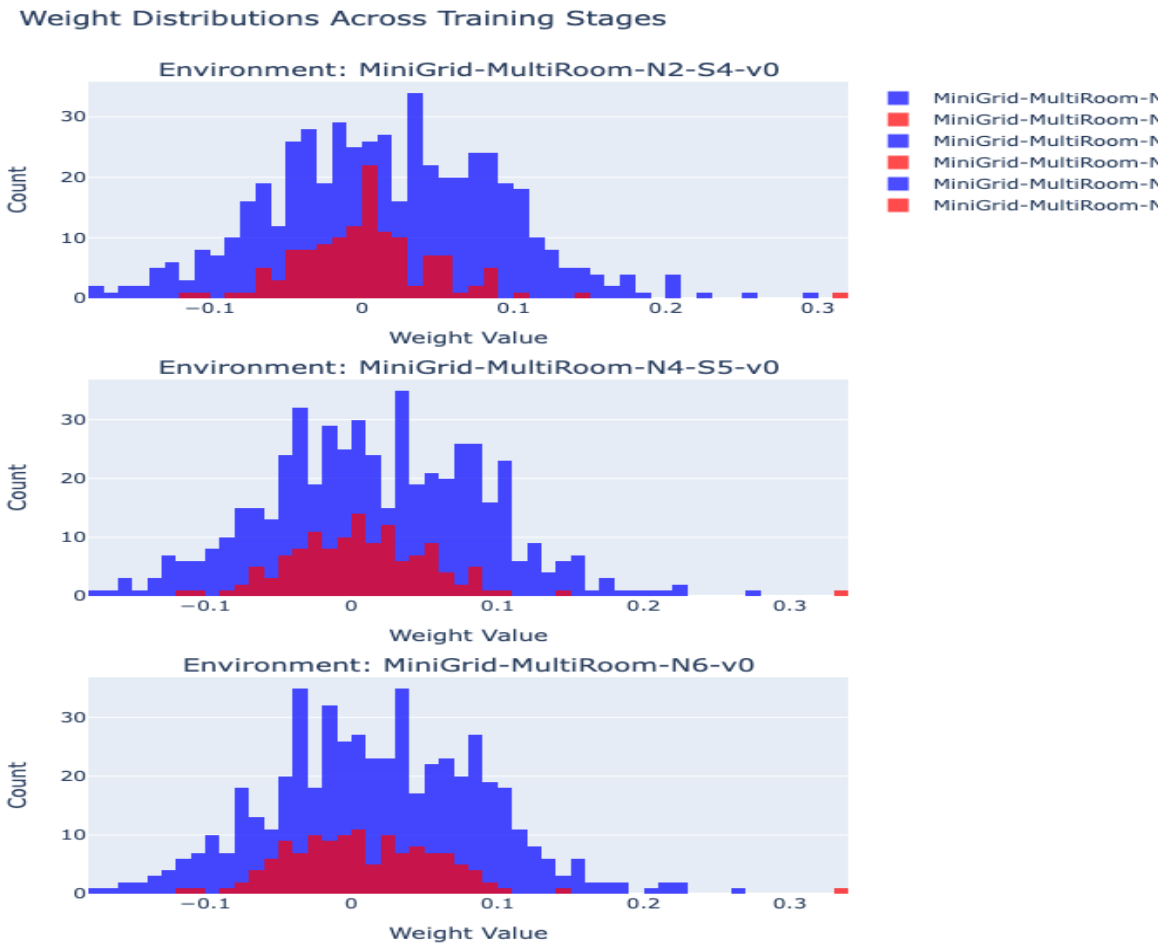


Fig. 9: Analysis Using ML-Flow -PPO Agent Entropy

Fig. 10: Analysis Using ML-Flow -PPO Agent Predicted Returns



11: Analysis Using ML-Flow -PPO


Weight Distributions Across Training Stages
Environment: MiniGrid-MultiRoom-N2-S4-v0
Environment: MiniGrid-MultiRoom-N4-S5-v0
Environment: MiniGrid-MultiRoom-N6-v0
Count
Weight Value
0
10
20
30
−0.1
0
0.1
0.2
0.3
MiniGrid-MultiRoom-N
MiniGrid-MultiRoom-N
MiniGrid-MultiRoom-N
MiniGrid-MultiRoom-N
MiniGrid-MultiRoom-N
MiniGrid-MultiRoom-N

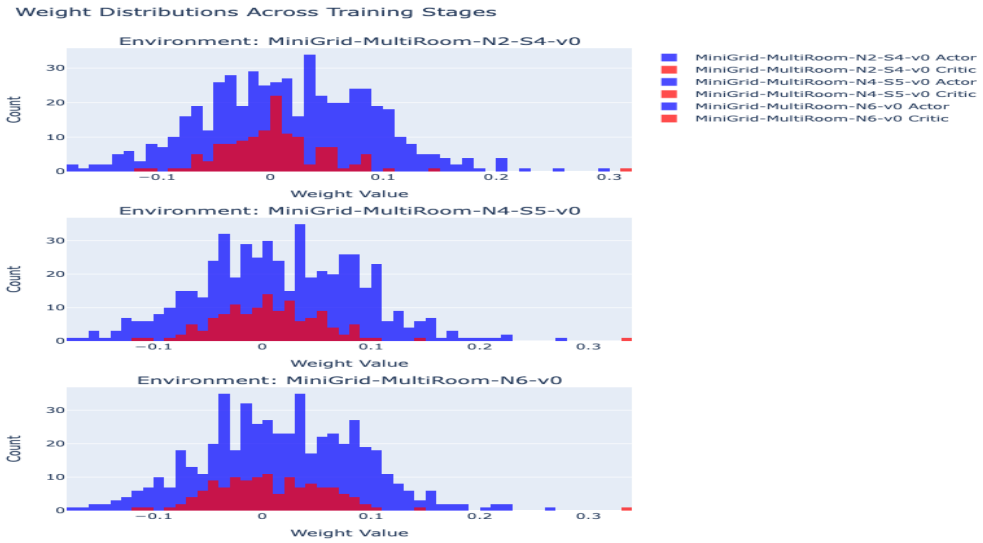


Fig. 12: Weight Dis. Analysis - PPO Agent